ST-ROBOT · TÀI LIỆU TỰ HỌC

Arduino Uno

Từ dòng điện, LED đến cảm biến siêu âm và cảnh báo khoảng cách.

6 bài học · Lý thuyết, ví dụ, thực hành, kiểm tra và tài liệu tham khảo

Lộ trình học tập

1. Mạch kín và LED
2. Tín hiệu số và nhịp chớp
3. Đo khoảng cách với HC-SR04
4. Dự án: cảnh báo vật cản
5. Hiệu chuẩn và sai số phép đo
6. Dự án: cảnh báo khoảng cách ổn định

Cách sử dụng tài liệu

Đọc mục tiêu và kiến thức nền; tự dự đoán trước ví dụ; thực hành theo các bước; ghi lại dữ liệu và giải thích sai lệch. Sau khi tự làm, đối chiếu đáp án và tự kiểm tra. Phòng lab bên ngoài không tự đồng bộ điểm; hoạt động mô phỏng không thay thế kiểm tra an toàn phần cứng thật.

Bài 1/6 · Mạch kín và LED

So sánh hai điện trở để thấy dòng qua LED thay đổi, rồi giải thích bằng một câu vì sao LED rời cần điện trở.

Chuẩn bị

Chưa cần biết lập trình hoặc công thức điện. Chuẩn bị giấy ghi hai cột "dự đoán" và "quan sát"; Ω là đơn vị điện trở, mA là đơn vị dòng điện. Bắt đầu bằng mô hình trong Lab, chưa cần kit thật.

Sau bài học, bạn có thể

1. So sánh hai giá trị dòng hiển thị khi chỉ thay điện trở.

2. Chỉ được đường kín và nêu vai trò hạn dòng của điện trở cho LED rời.

3. Phân biệt kết quả mô hình tính dòng với việc chạy chương trình Blink.

Cùng hiểu bài từ đầu

Một chiếc LED sáng lên nhờ có dòng điện chạy qua nó. Bài này giúp bạn nối ba ý: phải có đường đi kín, LED cần đúng chiều và dòng qua LED cần được giới hạn. Chưa cần thuộc công thức: trước hết hãy theo dõi điều gì thay đổi khi chỉ tăng điện trở, rồi dùng một phép tính nhỏ để giải thích.

Hãy hình dung một vòng đường có bơm nước, ống dẫn và một đoạn ống hẹp. Nước chỉ tuần hoàn khi đường ống không bị đứt; đoạn hẹp làm lưu lượng nhỏ hơn trong cùng điều kiện bơm.

Trong cách hình dung này, nguồn tạo chênh lệch điện thế để dòng chạy trong mạch kín; điện trở cản dòng. Điện trở không "tiêu hết dòng" trước khi dòng tới LED: các phần tử nối tiếp có cùng dòng điện ở trạng thái ổn định.

Giới hạn của ví von: Điện áp không phải nước và LED không phải ống thông thường. LED dẫn phụ thuộc chiều và điện áp theo quan hệ phi tuyến; ví von này chỉ giúp hiểu đường kín và vai trò hạn dòng.

Những từ cần hiểu

1. Điện áp · V: Chênh lệch điện thế giữa hai điểm. Nguồn 5 V mô tả chênh lệch này, không nói rằng mạch luôn có dòng 5 A.

2. Dòng điện · A, mA: Lượng điện tích đi qua một tiết diện mỗi giây. 1 A = 1 000 mA; 0,004 A là 4 mA.

3. Điện trở · Ω: Đại lượng liên hệ điện áp hai đầu điện trở với dòng qua nó: I = V/R. Cùng điện áp, điện trở lớn hơn cho dòng nhỏ hơn.

4. Điện áp rơi trên LED · Vf: Điện áp hai đầu LED khi đang dẫn ở một điều kiện cụ thể. Bài dùng 2 V như giả định để ước lượng, không coi đó là hằng số cho mọi LED.

Vì sao đổi 330 Ω thành 680 Ω làm dòng giảm?

Bắt đầu với: Mạch nối tiếp có nguồn 5 V, LED được mô hình hóa với Vf = 2 V. So sánh R = 330 Ω và R = 680 Ω; giữ nguyên nguồn và mô hình LED.

Tìm điện áp trên điện trở

Nguồn tạo 5 V; mô hình dành 2 V cho LED. Điện áp trên điện trở còn 5 − 2 = 3 V. Chính 3 V này được đưa vào công thức I = V/R.

Tính dòng ở lần đầu

Với 330 Ω: I = 3/330 A ≈ 0,00909 A. Đổi sang mA bằng cách nhân 1 000, được khoảng 9,09 mA.

Chỉ đổi một yếu tố

Với 680 Ω: I = 3/680 A ≈ 4,41 mA. So sánh 4,41 với 9,09: dòng giảm còn gần một nửa. Bạn đã kiểm tra tác động của điện trở, vì các giả định khác được giữ nguyên.

Kết quả: Hai đầu ra dự kiến là khoảng 9,09 mA và 4,41 mA. Đây là dòng tính từ mô hình, chưa phải số đo linh kiện thật.

Mẫu số R tăng làm I giảm khi điện áp trên điện trở giữ nguyên. Đó là lý do LED rời cần điện trở hạn dòng; thử nghiệm với LED tích hợp trên Uno không thay thế việc kiểm tra mạch LED rời.

Tự giải thích rồi đối chiếu

Nếu mạch bị hở, có thể dùng I = 3/R để kết luận LED vẫn sáng không?

Không. Công thức trên mô tả mạch nối tiếp đang kín trong mô hình đã nêu. Một đường dẫn bị đứt không có dòng ổn định chạy qua LED.

Vì sao phải viết mA sau số 4,41?

Đơn vị xác định độ lớn. 4,41 A lớn gấp 1 000 lần 4,41 mA; bỏ đơn vị dễ dẫn tới kết luận sai về dòng linh kiện.

Chọn độ sâu phù hợp

Mới bắt đầu · Đọc theo đường đi của dòng

Dùng ngón tay lần theo nguồn → điện trở → LED → đường về nguồn trên sơ đồ. Sau đó so sánh hai kết quả mA mà chưa cần tự biến đổi công thức.

Thử nói một câu: “Giữ nguồn và mô hình LED như cũ, tăng điện trở thì dòng…”.

Đã học cơ bản · Nối phép thử với định luật Ohm

Tách rõ điện áp nguồn 5 V và điện áp trên điện trở 3 V. Tính một hàng bằng A rồi đổi sang mA; đối chiếu với kết quả mô hình.

Tính cho 470 Ω. Bạn nên thu được khoảng 6,38 mA trong cùng giả định.

Muốn tìm hiểu sâu · Xét thêm công suất và sai số

Với 680 Ω, công suất điện trở là $P = I^2R \approx 0{,}0132$ W. Chọn linh kiện thật còn phải xét giới hạn dòng LED, công suất điện trở, sai số nguồn và Vf theo datasheet. Giới hạn tối đa của chân Uno không phải dòng thiết kế nên nhắm tới.

Giữ R = 680 Ω nhưng thay Vf giả định thành 1,8 V và 2,2 V. Tính hai dòng để thấy phép ước lượng phụ thuộc mô hình LED.

Kiến thức trọng tâm

Việc đầu tiên: thay một giá trị, đọc một kết quả

LED là đèn nhỏ chỉ sáng đúng cách khi dòng đi theo chiều phù hợp. Bàn thí nghiệm cho sẵn một mạch có nguồn, LED và điện trở. Điện trở hạn chế dòng qua LED: với cùng nguồn và cùng mô hình LED, tăng điện trở làm dòng nhỏ đi. Trước hết hãy kiểm tra điều đó bằng hai lần chạy; công thức ở phần đọc thêm giúp tính lại kết quả.

Đọc thêm · Đường đi của dòng điện

Dòng điện chỉ duy trì khi có nguồn và đường dẫn kín qua tải trở về nguồn. Điện áp là chênh lệch điện thế giữa hai điểm; dòng điện là lượng điện tích dịch chuyển qua một tiết diện trong một giây. Không dùng hai khái niệm này thay cho nhau.

Đọc thêm · LED cần hạn dòng

LED có cực tính và đặc tuyến không tuyến tính. Khi ước lượng sơ bộ, coi điện áp rơi trên LED là Vf rồi dùng I = (Vs − Vf)/R cho điện trở nối tiếp: Vs, Vf tính bằng V; R bằng Ω; I thu được bằng A. Đây là mô hình gần đúng, không thay datasheet. Với mạch giả định 5 V, Vf = 2 V, điện trở quyết định dòng; không suy ra dòng chỉ từ màu LED.

Đọc thêm · Từ mô phỏng tới Uno

LED_BUILTIN dùng LED có sẵn trên bo mạch theo thiết kế của bo. LED rời phải có điện trở riêng và đúng cực. Ngắt nguồn trước khi đổi dây; không nối chân OUTPUT trực tiếp với GND hay 5 V. Dòng tối đa của chân là giới hạn kỹ thuật, không phải dòng thiết kế mục tiêu. Không dùng chân Uno để cấp trực tiếp cho động cơ.

Ví dụ có lời giải

Với Vs = 5 V, Vf giả định = 2 V và R = 330 Ω: I = 3/330 A = 0,00909 A ≈ 9,1 mA. Công suất trên điện trở P = I²R ≈ 0,027 W. Nếu muốn I ≤ 10 mA trong đúng mô hình này thì R ≥ 300 Ω; 330 Ω là một giá trị thông dụng đáp ứng điều kiện. Phần cứng thật còn phải xét sai số và điện áp rơi thực.

Lỗi thường gặp và cách tránh

1. Không có điện trở thì LED không tự điều chỉnh về một dòng an toàn.

2. 0,009 A là 9 mA, không phải 0,009 mA; LED tích hợp không chứng minh mọi LED rời đều có điện trở.

Các bước thực hành

1. Dự đoán: khi đổi điện trở từ 330 Ω lên 1 000 Ω, dòng qua LED sẽ lớn hơn hay nhỏ hơn? Ghi lựa chọn trước khi mở kết quả.

2. Thao tác: vào Lab → Bàn thí nghiệm, nhập 330 Ω và chạy mô hình LED. Ghi giá trị dòng điện có đơn vị mA.

3. Quan sát: đổi riêng điện trở thành 1 000 Ω, chạy lại và ghi dòng điện. Giữ nguyên giả định nguồn 5 V và LED 2 V.

4. Giải thích: đối chiếu hai số với dự đoán, viết “Điện trở lớn hơn làm dòng ... khi giữ nguyên mô hình”. Dùng sơ đồ nguồn → điện trở → LED → đường về nguồn để chỉ đường kín.

5. Thử tiếp: nhập 220 Ω, dự đoán trước rồi đối chiếu. Nếu muốn chạy Blink, dùng editor Arduino ở Lab và LED tích hợp; bàn mô hình LED chỉ tính dòng theo đầu vào, không thực thi đoạn C++.

6. Mở rộng khi đã làm được: đọc công thức hạn dòng và tự tính lại một hàng. Nếu dùng LED rời thật, ngắt nguồn trước khi đổi dây và nhờ người hướng dẫn kiểm tra điện trở, cực tính.

Mã mẫu · Arduino C++ · LED tích hợp

```
void setup() { pinMode(LED_BUILTIN, OUTPUT); }
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); delay(500);
  digitalWrite(LED_BUILTIN, LOW); delay(500);
}
```

Chọn và sử dụng phòng lab

Bắt đầu ở Bàn thí nghiệm của bài LED: dự đoán, nhập lần lượt 330 Ω và 1 000 Ω, chạy rồi ghi số mA. Mô hình dùng I = (5 − 2)/R, chỉ tính từ các giá trị đầu vào. Muốn thử chương trình Blink, chuyển sang editor Arduino phù hợp trong Lab, dán mẫu C++ và chạy LED_BUILTIN: sáng 500 ms, tắt 500 ms. CircuitJS dùng để khảo sát mạch; LED tích hợp và LED rời có cách mắc khác nhau.

Bài tập áp dụng

Lập bảng cho 330 Ω, 1 000 Ω và 220 Ω: dự đoán dòng tăng/giảm, dòng mô hình hiển thị và một câu giải thích. Trong mô hình 5 V, LED 2 V, chọn giá trị có dòng không quá 10 mA. Phần thêm: tự tính một hàng bằng công thức trong mục đọc thêm.

Gợi ý đối chiếu sau khi tự làm

Mô hình cho 330 Ω → khoảng 9,09 mA; 1 000 Ω → 3 mA; 220 Ω → khoảng 13,64 mA. Điện trở tăng làm dòng giảm trong cùng mô hình. 330 Ω và 1 000 Ω đáp ứng không quá 10 mA. Phép tính thêm: I = (5 − 2)/330 A ≈ 0,00909 A = 9,09 mA. Linh kiện thật có điện áp rơi, dòng định mức và sai số riêng.

Tự kiểm tra mức hiểu

1. Khi đổi 330 Ω thành 1 000 Ω, em dự đoán gì và mô hình hiển thị gì?

2. Vì sao không dùng kết quả của LED tích hợp để bỏ điện trở của LED rời?

Câu hỏi trắc nghiệm

Giữ nguyên điện áp, tăng điện trở làm dòng điện...

1. Giảm

2. Tăng

3. Không đổi

Đáp án: 1. Với điện áp trên điện trở không đổi và R > 0, I = V/R. R tăng thì I giảm; ở đây V = Vs − Vf trong mô hình LED gần đúng.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

Arduino — chân tín hiệu số

https://docs.arduino.cc/learn/microcontrollers/digital-pins/

Arduino — ví dụ Blink

https://docs.arduino.cc/built-in-examples/basics/Blink/

Bài 2/6 · Tín hiệu số và nhịp chớp

Tính chu kỳ, tần số và tỷ lệ bật của LED; nhận biết giới hạn của lệnh chờ.

Chuẩn bị

Đã biết OUTPUT, HIGH/LOW và chạy ví dụ LED_BUILTIN của bài LED.

Sau bài học, bạn có thể

1. Đọc đúng đơn vị ms và đổi chu kỳ sang giây.

2. Tính T, f và duty cycle từ hai khoảng bật/tắt.

3. Giải thích vì sao delay không phù hợp khi cần xử lý nhiều nhiệm vụ liên tục.

Cùng hiểu bài từ đầu

Một đèn nhấp nháy có ít nhất hai điều để mô tả: bao lâu lặp lại một lần và trong lần đó sáng bao lâu. Tách hai điều này giúp bạn đọc chương trình theo thời gian, thay vì chỉ nhìn đèn rồi nói "nhanh" hoặc "chậm".

Hãy vỗ tay rồi nghỉ theo một nhịp đều. Một nhịp gồm cả đoạn vỗ và đoạn nghỉ; chỉ đo đoạn vỗ sẽ thiếu một phần của nhịp.

LED cũng có đoạn bật và đoạn tắt. Chu kỳ là khoảng thời gian từ một lần bắt đầu bật tới lần bắt đầu bật kế tiếp. Tỷ lệ bật cho biết phần thời gian bật trong cả chu kỳ.

Giới hạn của ví von: Tay người không tạo mức logic chính xác và cảm nhận ánh sáng không giống việc đếm nhịp. Ví von này không dự đoán độ sáng cảm nhận hay dòng điện của LED.

Những từ cần hiểu

1. Chu kỳ · T: Thời gian cho một lần lặp đầy đủ. Nếu bật 300 ms rồi tắt 700 ms, T = 1 000 ms.

2. Tần số · Hz: Số chu kỳ trong một giây. 1 Hz là một chu kỳ mỗi giây, không phải một mili giây.

3. Tỷ lệ bật · duty cycle: Thời gian bật chia tổng thời gian chu kỳ, thường viết bằng phần trăm.

4. Lệnh chờ · delay: Tạm dừng việc chạy tiếp các lệnh trong luồng chính. Trong thời gian đó, những lệnh đọc cảm biến đặt phía sau chưa được thực hiện.

Hai nhịp cùng 1 Hz nhưng tỷ lệ bật khác nhau

Bắt đầu với: Phương án A bật 300 ms, tắt 700 ms. Phương án B bật 600 ms, tắt 400 ms. Một giây bằng 1 000 ms.

Ghép đủ một chu kỳ

A có T = 300 + 700 = 1 000 ms. B có T = 600 + 400 = 1 000 ms. Không lấy riêng thời gian bật làm chu kỳ.

Đổi chu kỳ thành tần số

Cả hai có T = 1 s nên f = 1/T = 1 Hz. Nếu giữ ms, dùng f = 1 000/T, cũng được 1 Hz.

Tính phần thời gian bật

A có D = 300/1 000 × 100% = 30%. B có D = 600/1 000 × 100% = 60%. B bật lâu hơn trong mỗi chu kỳ, dù số chu kỳ mỗi giây bằng nhau.

Kết quả: Cùng tần số 1 Hz; tỷ lệ bật lần lượt 30% và 60%. Bạn đã dùng hai đại lượng khác nhau để mô tả cùng một tín hiệu.

Khi đọc mã, hãy dựng một trục thời gian: bật → chờ → tắt → chờ → lặp. Cách này còn cho biết chương trình có thể chậm phản hồi nút bấm vì đang chờ ở đâu.

Tự giải thích rồi đối chiếu

Bật 250 ms, tắt 250 ms có phải 4 Hz không?

Không. Cả chu kỳ là 500 ms = 0,5 s, nên tần số là 2 Hz. 250 ms chỉ là nửa chu kỳ.

Đang delay 700 ms, chương trình có kiểm tra nút bằng lệnh nằm sau delay ngay lập tức không?

Không. Lệnh phía sau phải chờ luồng chính chạy tiếp. Khi cần phản hồi thường xuyên, có thể so sánh thời gian millis giữa các lượt lặp thay cho chờ dài.

Chọn độ sâu phù hợp

Mới bắt đầu · Vẽ trước khi tính

Vẽ một thanh dài 1 000 ms, tô phần bật 300 ms, để trắng phần tắt 700 ms. Bạn có thể nhìn thấy tỷ lệ 3 phần bật trong 10 phần của chu kỳ.

Chỉ ra điểm đầu và điểm cuối của một chu kỳ trên hình.

Đã học cơ bản · Giữ một đại lượng, đổi đại lượng kia

So sánh 300/700 với 600/400 để đổi duty mà giữ tần số. So sánh 300/700 với 600/1 400 để giữ duty 30% nhưng giảm tần số còn 0,5 Hz.

Thiết kế một nhịp 2 Hz có tỷ lệ bật 20%; kiểm tra tổng thời gian và đơn vị.

Muốn tìm hiểu sâu · Lập lịch không chặn bằng thời gian

Tách trạng thái đèn và thời điểm cần đổi trạng thái. Mỗi lượt loop đọc cảm biến rồi so sánh thời gian đã trôi với khoảng bật/tắt tương ứng. Khi triển khai millis, dùng hiệu thời gian với kiểu số nguyên không dấu phù hợp để xử lý bộ đếm quay vòng.

Mô tả điều gì xảy ra nếu xử lý cảm biến tốn lâu hơn khoảng bật dự kiến; phân biệt thời gian yêu cầu và thời gian chuyển thực tế.

Kiến thức trọng tâm

Mức logic và thời gian

HIGH và LOW là hai mức logic mà phần mềm yêu cầu ở chân OUTPUT. Không hiểu HIGH là một dòng điện cố định. Đối với đoạn nháy LED, một chu kỳ đầy đủ gồm thời gian bật t_on rồi thời gian tắt t_off; phải đo giữa hai trạng thái cùng pha.

Ba đại lượng khác nhau

T = t_on + t_off. Nếu T đo bằng ms và T > 0 thì f = 1000/T (Hz); nếu T đo bằng s thì f = 1/T. Duty cycle D = t_on/T × 100%. Đổi D có thể đổi tỷ lệ thời gian sáng nhưng không nhất thiết đổi tần số. Nháy chậm có thể quan sát thành từng lần; không kết luận độ sáng cảm nhận chỉ bằng một khung hình mô phỏng.

Lệnh chờ và xử lý đồng thời

delay(200) tạm dừng luồng chạy chính khoảng 200 ms. Nó không tắt nguồn bo mạch và một số cơ chế hệ thống vẫn hoạt động, nhưng các lệnh tiếp theo trong loop chưa được thực hiện. Khi phải đọc nút, cảm biến và nháy LED cùng lúc, dùng cách so sánh thời gian millis thay cho một chuỗi chờ dài.

Ví dụ có lời giải

Bật 200 ms, tắt 800 ms: T = 1000 ms = 1 s, f = 1 Hz, D = 20%. Bật 500 ms, tắt 500 ms vẫn 1 Hz nhưng D = 50%. Trong 10 s có xấp xỉ 10 chu kỳ; cách đếm số cạnh sáng ở đúng mốc đầu/cuối có thể khác 1 nên phải ghi quy ước đo.

Lỗi thường gặp và cách tránh

1. Hai nhịp 200/800 và 500/500 có cùng chu kỳ nhưng khác duty cycle.

2. Tần số có đơn vị Hz; 1000/T chỉ đúng khi T đang tính bằng mili giây.

Các bước thực hành

1. Lập bảng dự đoán T, f, D cho 200/800 ms, 500/500 ms và 100/400 ms.

2. Chạy code mẫu trên LED tích hợp; đo nhiều chu kỳ bằng đồng hồ thay vì chỉ một lần.

3. Giữ T = 1000 ms và đổi t_on để đạt D = 25%.

4. Thử thêm một thao tác cần phản hồi trong lúc delay dài; nêu khoảng thời gian có thể phải chờ.

5. Ghi kết quả và đề xuất dùng millis khi cần nhiều nhiệm vụ; không khẳng định delay vô hiệu hóa mọi hoạt động của bo.

Mã mẫu · Arduino C++ · LED tích hợp

```
void setup() { pinMode(LED_BUILTIN, OUTPUT); }
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); delay(200);
  digitalWrite(LED_BUILTIN, LOW); delay(800);
}
```

Chọn và sử dụng phòng lab

Editor Arduino phù hợp chạy được code LED_BUILTIN. Bàn thí nghiệm trong trang giúp tính nhịp; nếu hoạt ảnh không đồng bộ thời gian thực, dùng kết quả công thức để đối chiếu và đo nhiều chu kỳ trên thiết bị thật.

Bài tập áp dụng

Thiết kế nhịp 1 Hz với duty cycle 25%. Sau đó thiết kế nhịp 2 Hz nhưng vẫn 25%; viết cả hai cặp thời gian.

Gợi ý đối chiếu sau khi tự làm

1 Hz: T = 1000 ms, bật 250 ms, tắt 750 ms. 2 Hz: T = 500 ms, bật 125 ms, tắt 375 ms. Cùng tỷ lệ bật không có nghĩa cùng tốc độ nháy.

Tự kiểm tra mức hiểu

1. Nhịp 100/400 ms có tần số và duty cycle bao nhiêu?

2. Vì sao gọi delay(1000) giữa hai lần đọc nút có thể làm phản hồi chậm?

Câu hỏi trắc nghiệm

Bật 200 ms, tắt 800 ms thì một chu kỳ dài...

1. 200 ms

2. 800 ms

3. 1000 ms

Đáp án: 3. Một chu kỳ chứa cả phần bật và phần tắt: T = 200 + 800 = 1000 ms. Đây là thời gian lặp lại, không phải riêng thời gian LED sáng.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

Arduino — ví dụ Blink

https://docs.arduino.cc/built-in-examples/basics/Blink/

Arduino — Blink Without Delay

https://docs.arduino.cc/built-in-examples/digital/BlinkWithoutDelay/

Bài 3/6 · Đo khoảng cách với HC-SR04

Giải thích phép đo siêu âm khứ hồi, tính khoảng cách có đơn vị và xử lý timeout.

Chuẩn bị

Biết tín hiệu số, đơn vị ms/µs và dùng Serial Monitor; lắp phần cứng khi có giáo viên kiểm tra.

Sau bài học, bạn có thể

1. Mô tả chức năng riêng của Trig và Echo.

2. Tính d = v × t/2 với đơn vị nhất quán.

3. Đánh dấu phép đo mất Echo là không hợp lệ thay vì khoảng cách 0 cm.

Cùng hiểu bài từ đầu

Cảm biến siêu âm không đọc trực tiếp một con số "cm". Nó giúp chương trình đo một khoảng thời gian. Muốn từ thời gian suy ra khoảng cách, bạn cần biết sóng đi bao nhanh và nhớ rằng sóng đã đi cả lượt tới lẫn lượt về.

Bạn gọi lớn trước một vách núi và đợi tiếng vọng. Thời gian chờ chứa cả đoạn âm đi tới vách và đoạn âm quay lại tai.

HC-SR04 cũng dựa trên một hành trình âm thanh khứ hồi. Chương trình dùng độ rộng xung Echo như dữ liệu thời gian rồi tính khoảng cách một chiều tới vật.

Giới hạn của ví von: Trig và Echo là tín hiệu điện giữa bo và module, không phải âm thanh đi trong không khí. Việc có vật trước mặt chưa bảo đảm luôn nhận được phản xạ đủ tốt.

Những từ cần hiểu

1. Siêu âm: Âm thanh có tần số vượt vùng nghe thông thường của người. Module phát sóng và nhận phản xạ.

2. Thời gian khứ hồi · µs: Thời gian đi tới vật rồi quay về; 1 µs = một phần triệu giây.

3. Tốc độ âm · cm/µs: Trong phép tính này lấy gần đúng 0,034 cm/µs. Con số thay đổi theo môi trường, đặc biệt nhiệt độ.

4. Timeout: Hết thời gian chờ mà chưa có xung đo hoàn chỉnh. Đây là dữ liệu lỗi, không phải khoảng cách bằng 0.

Từ 1 500 µs đến khoảng cách một chiều

Bắt đầu với: Một xung Echo hợp lệ dài 1 500 µs. Dùng tốc độ âm gần đúng 0,034 cm/µs; chưa xét nhiệt độ và sai số phản xạ.

Tính toàn bộ đường sóng đi

Nhân 0,034 cm/µs với 1 500 µs, được 51 cm. Đơn vị µs triệt tiêu; 51 cm là tổng hành trình đi và về.

Lấy một nửa hành trình

Khoảng cách từ cảm biến tới vật là 51/2 = 25,5 cm. Bỏ phép chia 2 sẽ khiến kết quả lớn gấp đôi trong mô hình này.

Xử lý một lượt không có Echo

Nếu lượt tiếp theo trả 0 do timeout, ghi "không đo được". Không tính d = 0 cm và không trộn mẫu lỗi với 25,5 cm để lấy trung bình.

Kết quả: Lượt hợp lệ cho 25,5 cm. Lượt timeout không tạo khoảng cách mới; đầu ra cần kèm trạng thái dữ liệu.

Một số đo tốt gồm cả giá trị và điều kiện hợp lệ. Nhờ vậy, chương trình phía sau phân biệt được "vật gần" và "cảm biến chưa cung cấp dữ liệu".

Tự giải thích rồi đối chiếu

Echo dài gấp đôi thì khoảng cách trong mô hình thay đổi thế nào?

Khoảng cách cũng gấp đôi, vì d = 0,034 × t/2 là quan hệ tuyến tính khi giữ tốc độ âm như cũ và xung hợp lệ.

Vì sao một tấm vải nghiêng có thể khó đo hơn một mặt phẳng cứng vuông góc?

Vật liệu có thể hấp thụ âm và góc nghiêng có thể phản xạ âm ra khỏi cảm biến. Công thức không sửa được một phản xạ yếu hoặc sai mục tiêu.

Chọn độ sâu phù hợp

Mới bắt đầu · Theo dõi lượt đi và lượt về

Vẽ cảm biến và vật, thêm hai mũi tên ngược chiều. Viết 51 cm trên tổng hai mũi tên rồi chia đều để thấy vì sao cần chia 2.

Tính nhẩm cho Echo = 1 000 µs và so với ví dụ.

Đã học cơ bản · Kiểm tra đơn vị trước công thức

Có thể dùng 340 m/s với thời gian tính bằng s hoặc 0,034 cm/µs với thời gian tính bằng µs. Hai cách phải cho cùng khoảng cách sau khi đổi đơn vị.

Đổi 1 500 µs thành giây rồi tính lại bằng m/s.

Muốn tìm hiểu sâu · Tách sai số mô hình và lỗi đọc

Thay đổi tốc độ âm giả định làm dịch khoảng cách tính được ngay cả khi thời gian đúng. Timeout lại là trường hợp không có mẫu hợp lệ. Hai vấn đề cần hai cách xử lý khác nhau; lấy trung bình không tự chữa mọi lỗi. Echo 5 V của module cần chuyển mức phù hợp khi nối sang bo chỉ nhận 3,3 V.

Tính kết quả với v = 0,033 và 0,035 cm/µs ở cùng 1 500 µs; nêu phạm vi chênh lệch do riêng giả định tốc độ âm.

Kiến thức trọng tâm

Nguyên lý đo thời gian

Module HC-SR04 phát một đợt siêu âm sau xung kích Trig; Echo là tín hiệu điện mà chương trình đo độ rộng để suy ra thời gian sóng đi tới vật rồi quay lại. Trig và Echo không trực tiếp là sóng âm. Nguồn nuôi và mức logic phải phù hợp bo đang dùng.

Tính khoảng cách và điều kiện áp dụng

Với tốc độ âm xấp xỉ v = 340 m/s = 0,034 cm/µs, d = v × t/2. t là thời gian khứ hồi tính bằng µs, nên d thu được bằng cm. Tốc độ âm phụ thuộc môi trường, đặc biệt nhiệt độ; góc nghiêng, vật mềm/hẹp và nhiều mặt phản xạ có thể gây số đo sai. Đây không phải công thức đảm bảo mọi vật đều đo được.

Timeout là trạng thái dữ liệu

pulseIn(echo, HIGH, 30000UL) giới hạn thời gian chờ. Kết quả 0 nghĩa không đo được xung hoàn chỉnh trong thời gian chờ; phải hiển thị lỗi hoặc bỏ mẫu, không đưa vào trung bình như 0 cm. Lấy mẫu cách nhau khoảng 100 ms trong ví dụ. Mạch minh họa dùng Uno 5 V; không nối Echo 5 V trực tiếp sang bo chỉ nhận logic 3,3 V khi chưa có chuyển mức phù hợp.

Ví dụ có lời giải

Echo = 1000 µs cho d = 0,034 × 1000/2 = 17 cm. Echo = 2000 µs cho 34 cm. Ngược lại, vật cách 20 cm cho thời gian lý tưởng khoảng 2 × 20/0,034 ≈ 1176 µs. Giá trị timeout 0 là “không có số đo hợp lệ”, không phải một vật chạm sát cảm biến.

Lỗi thường gặp và cách tránh

1. Quên chia 2 sẽ tính quãng đường đi–về thay vì khoảng cách tới vật.

2. Timeout không chứng minh không có vật cản; không thể gọi đó là trạng thái an toàn.

Các bước thực hành

1. Trong mô phỏng hỗ trợ HC-SR04, đặt VCC 5 V, GND chung, Trig D9, Echo D10 trên Uno. Với kit thật, ngắt nguồn và kiểm tra sơ đồ module trước khi nối.

2. Tính trước thời gian khứ hồi lý tưởng ở 10, 20 và 40 cm.

3. Chạy code, mở Serial Monitor 9600 baud; đối chiếu số đo ở từng vị trí.

4. Thử trường hợp không nhận phản hồi hoặc ngoài vùng đo; kiểm tra thông báo No echo.

5. Ghi nhiệt độ/điều kiện và sai khác; không quy mọi lỗi thành một khoảng cách thật.

Mã mẫu · Arduino C++ · HC-SR04 trên Uno 5V

```
const int trig=9, echo=10;
void setup(){
 Serial.begin(9600); pinMode(trig,OUTPUT); pinMode(echo,INPUT);
}
void loop(){
 digitalWrite(trig,LOW); delayMicroseconds(2);
 digitalWrite(trig,HIGH); delayMicroseconds(10);
 digitalWrite(trig,LOW);
 unsigned long t=pulseIn(echo,HIGH,30000UL);
 if(t==0) Serial.println("No echo");
 else Serial.println(t*0.034/2);
 delay(100);
}
```

Chọn và sử dụng phòng lab

Dùng bàn thí nghiệm để tính quan hệ Echo–khoảng cách và kiểm tra ngưỡng. Chạy code cảm biến trong môi trường có module HC-SR04 hoặc trên kit Uno đã lắp đúng. Chọn AVR8js LED cho bài đèn; chọn môi trường có HC-SR04 cho bài đo siêu âm.

Bài tập áp dụng

Tính d cho Echo 1000 µs và 2000 µs. Thiết kế thông báo riêng cho timeout; nêu hai điều kiện có thể làm phép đo sai.

Gợi ý đối chiếu sau khi tự làm

17 cm và 34 cm với v giả định 0,034 cm/µs. Với t = 0, hiển thị không có số đo hợp lệ và không dùng để ra quyết định như khoảng cách 0. Ví dụ sai số: nhiệt độ khác, góc mặt phản xạ nghiêng, vật hấp thụ âm hoặc nhiễu phản xạ.

Tự kiểm tra mức hiểu

1. Tại sao phải đổi 340 m/s thành 0,034 cm/µs trong code?

2. Vì sao vật nghiêng hoặc bọc vải có thể cho kết quả khác vật phẳng cứng?

Câu hỏi trắc nghiệm

Vì sao công thức khoảng cách chia cho 2?

1. Có hai chân tín hiệu

2. Sóng đi và về

3. Nguồn điện 5V

Đáp án: 2. Sóng truyền từ cảm biến đến vật và quay lại cảm biến. t đo toàn hành trình nên khoảng cách một chiều bằng v × t/2.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

Arduino — pulseIn và giá trị timeout

https://docs.arduino.cc/language-reference/en/functions/advanced-io/pulseIn/

Arduino — ví dụ đo siêu âm Ping

https://docs.arduino.cc/built-in-examples/sensors/Ping/

Bài 4/6 · Dự án: cảnh báo vật cản

Xây dựng cảnh báo một ngưỡng có ba trạng thái: gần, xa và lỗi đo.

Chuẩn bị

Đã chạy bài HC-SR04, hiểu kiểu dữ liệu hợp lệ/không hợp lệ và cấu trúc if/else.

Sau bài học, bạn có thể

1. Chuyển yêu cầu bằng lời thành điều kiện d < 20 cm.

2. Dự đoán chính xác kết quả tại 19, 20 và 21 cm.

3. Tách lỗi đo khỏi kết luận vật ở gần hoặc xa.

Cùng hiểu bài từ đầu

Sau khi có số đo, robot còn phải quyết định làm gì. Một quy tắc cảnh báo tốt cần nói rõ ba điều: khi nào dữ liệu dùng được, giá trị nào thuộc nhánh gần và khi lỗi thì báo ra sao. Bạn có thể kiểm tra quy tắc này trên giấy trước khi chạy.

Ở cổng soát vé, người kiểm tra cần biết vé đọc được hay không trước khi xét vé còn hạn. Không đọc được mã không đồng nghĩa với có vé hợp lệ.

Tương tự, chương trình kiểm tra trạng thái cảm biến trước, rồi mới so khoảng cách với ngưỡng. Tách hai bước giúp tránh coi lỗi đo là một khoảng cách thật.

Giới hạn của ví von: Chính sách soát vé không quyết định chính sách an toàn cho robot. Việc bật LED trong bài chỉ biểu diễn trạng thái phần mềm, không bảo đảm dừng được một cơ cấu chuyển động.

Những từ cần hiểu

1. Ngưỡng · T: Giá trị dùng làm ranh giới ra quyết định; bài chọn T = 20 cm.

2. Điều kiện d < T: Đúng khi khoảng cách nhỏ hơn ngưỡng. Dấu < không bao gồm trường hợp bằng ngưỡng.

3. Ca kiểm thử ở biên: Đầu vào ngay dưới, đúng bằng và ngay trên ranh giới; giúp phát hiện chọn sai dấu so sánh.

4. Chính sách lỗi: Quy tắc xử lý khi dữ liệu không dùng được. Ví dụ của bài ghi SENSOR_ERROR và bật LED để gây chú ý.

Đọc một bảng quyết định đủ cả dữ liệu lỗi

Bắt đầu với: T = 20 cm, dùng điều kiện d < 20. Bốn lượt đầu vào: hợp lệ 18 cm; hợp lệ 20 cm; hợp lệ 22 cm; timeout không có d.

Tách lượt không hợp lệ

Lượt timeout được xử lý riêng: ghi SENSOR_ERROR và bật LED theo chính sách của bài. Không đưa một số d giả vào phép so sánh.

So sánh ba khoảng cách thật

18 < 20 đúng nên chọn NEAR. 20 < 20 sai và 22 < 20 sai nên cả hai chọn FAR. Mỗi số d đều dùng cùng đơn vị cm với T.

Ghi cả nhãn và đèn

18 cm → NEAR, LED sáng; 20 và 22 cm → FAR, LED tắt; timeout → SENSOR_ERROR, LED sáng. Hai nhãn khác nhau có thể cùng làm LED sáng.

Kết quả: Có ba trạng thái cần hiểu: gần, xa theo phép đo và lỗi cảm biến. Chỉ nhìn một đèn sáng chưa phân biệt được gần với lỗi.

Bảng dự kiến cho bạn một chuẩn để đối chiếu chương trình. Nếu 20 cm lại thành NEAR, hãy kiểm tra có dùng ≤ thay cho < hoặc có đổi đơn vị sai không.

Tự giải thích rồi đối chiếu

Nếu muốn cả 20 cm cũng cảnh báo gần, cần đổi gì?

Đổi yêu cầu thành d ≤ 20 và cập nhật kết quả dự kiến ở đúng 20 cm. Đây là thay đổi hành vi, không chỉ thay cách viết.

Vì sao thử riêng 5 cm và 80 cm chưa đủ kiểm tra ngưỡng?

Cả < và ≤ đều cho cùng nhánh ở hai số đó. Chúng chỉ khác ở đúng ngưỡng; cần thêm ca biên và ca lỗi.

Chọn độ sâu phù hợp

Mới bắt đầu · Đọc điều kiện thành câu

Đọc d < 20 là "khoảng cách nhỏ hơn 20 cm". Hãy tự hỏi câu đó đúng hay sai với từng thẻ 18, 20, 22; đừng bắt đầu bằng nhớ cú pháp.

Làm bảng hai cột đầu vào và trạng thái dự kiến trước khi mở Lab.

Đã học cơ bản · Liên kết yêu cầu, điều kiện và kiểm thử

Một yêu cầu chính xác phải có đơn vị, dấu so sánh và hành vi lỗi. Ghi cả nhãn trạng thái và LED để không nhầm biểu hiện với nguyên nhân.

Nếu dữ liệu đầu vào đổi sang mm, hãy viết lại ngưỡng tương ứng và ba ca biên.

Muốn tìm hiểu sâu · Phân biệt đặc tả logic và bảo đảm thực tế

Một bảng quyết định đúng không chứng minh cảm biến luôn chính xác hoặc robot dừng kịp. Cần đánh giá thời gian lấy mẫu, độ trễ, khoảng dừng và khả năng nhận biết lỗi theo thiết bị thực. Bài này chỉ kiểm tra logic cảnh báo.

Mô tả một phép thử mà LED đúng theo số đo nhưng vật thực tế vẫn ở vị trí khác vì cảm biến đọc sai.

Kiến thức trọng tâm

Từ phép đo đến quyết định

Một chu trình có ba bước: đọc dữ liệu → kiểm tra hợp lệ → so sánh với ngưỡng. Trước khi viết code, quy định rõ trạng thái mong muốn và đơn vị. Trong bài này: d < 20 cm là gần; d ≥ 20 cm là xa theo phép đo; mất Echo là lỗi riêng.

Kiểm thử ngay tại biên

Dấu < không giống ≤. Với ngưỡng 20, giá trị đúng 20 không thỏa d < 20. Bộ ca nhỏ nhất nên gồm dưới ngưỡng, bằng ngưỡng, trên ngưỡng và lỗi dữ liệu. Dự đoán trước sẽ giúp phát hiện lỗi điều kiện, thay vì chỉ thử các giá trị thuận lợi.

Phân biệt mô hình và bảo đảm an toàn

LED chỉ biểu diễn một quyết định phần mềm. Nếu cảm biến lỗi, code mẫu giữ LED sáng như một dấu hiệu cần chú ý và in SENSOR_ERROR; đèn sáng vì lỗi không đồng nghĩa đã đo được vật ở gần. Đây là bài học cảnh báo, không phải thiết bị bảo vệ hay cơ chế đảm bảo robot sẽ dừng kịp.

Ví dụ có lời giải

Bảng kỳ vọng: (hợp lệ,19 cm) → NEAR và LED sáng; (hợp lệ,20 cm) → FAR và LED tắt; (hợp lệ,21 cm) → FAR; (không hợp lệ,không có d) → SENSOR_ERROR và LED sáng theo chính sách lỗi của ví dụ. Không gán một khoảng cách giả cho dòng cuối.

Lỗi thường gặp và cách tránh

1. Ở đúng 20 cm, điều kiện d < 20 là sai, không phải gần đúng bằng nên được bật.

2. LED sáng do lỗi không phải bằng chứng cảm biến vừa đo được vật cản.

Các bước thực hành

1. Viết bốn dòng bảng kiểm thử 19/20/21 cm và mất Echo trước khi chạy.

2. Giữ mạch HC-SR04 của bài trước; dùng LED_BUILTIN để không cần thêm mạch LED ngoài.

3. Chạy sketch đầy đủ và đối chiếu cả Serial lẫn LED.

4. Sửa điều kiện < thành <= chỉ trong một bản thử; chỉ ra ca nào đổi kết quả.

5. Ghi vì sao thông báo lỗi phải phân biệt với cảnh báo gần và vì sao chưa thể kết luận tránh va chạm.

Mã mẫu · Arduino C++ · sketch đầy đủ cho Uno + HC-SR04

```
const byte TRIG = 9;
const byte ECHO = 10;

bool readDistance(float &cm) {
 digitalWrite(TRIG, LOW); delayMicroseconds(2);
 digitalWrite(TRIG, HIGH); delayMicroseconds(10);
 digitalWrite(TRIG, LOW);
 unsigned long us = pulseIn(ECHO, HIGH, 30000UL);
 if (us == 0) return false;
 cm = us * 0.034f / 2.0f;
 return true;
}

void setup() {
 Serial.begin(9600);
 pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
 pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
 float cm = 0;
 bool valid = readDistance(cm);
 if (!valid) {
  digitalWrite(LED_BUILTIN, HIGH); // Chinh sach loi cua vi du
  Serial.println("SENSOR_ERROR");
 } else {
  bool near = cm < 20.0f;
  digitalWrite(LED_BUILTIN, near ? HIGH : LOW);
  Serial.print(cm); Serial.println(near ? " cm NEAR" : " cm FAR");
 }
 delay(100);
}
```

Chọn và sử dụng phòng lab

Dùng bàn thí nghiệm để lập bảng quyết định; code trên dành cho Uno có HC-SR04. Mini lab chỉ mô tả quan hệ số đo/ngưỡng; hãy ghi riêng phép thử lỗi vì không phải mọi thanh trượt đều tạo được timeout.

Bài tập áp dụng

Hoàn thiện bảng 19/20/21 cm + timeout, rồi đề xuất quy tắc ngưỡng 15 cm. Giữ thông báo lỗi khác với hai trạng thái khoảng cách.

Gợi ý đối chiếu sau khi tự làm

Với < 20: gần, xa, xa; timeout là SENSOR_ERROR. Với < 15, cần thử 14/15/16 cm để kiểm tra đúng biên mới. Chính sách đèn khi lỗi phải được mô tả rõ, không gọi lỗi là một phép đo gần hay xa.

Tự kiểm tra mức hiểu

1. Thay < bằng <= làm đổi kết quả của ca 19, 20 hay 21 cm?

2. Nếu bỏ bước kiểm tra valid, timeout 0 có thể bị hiểu sai như thế nào?

Câu hỏi trắc nghiệm

Với d < 20, khi d = 20 cm thì...

1. Cảnh báo bật

2. Cảnh báo tắt

3. Chắc chắn lỗi cảm biến

Đáp án: 2. 20 < 20 là sai nên LED cảnh báo khoảng cách tắt ở dữ liệu hợp lệ đúng 20 cm. Đây là kết quả của dấu so sánh đã chọn.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

Arduino — pulseIn và giá trị timeout

https://docs.arduino.cc/language-reference/en/functions/advanced-io/pulseIn/

Arduino — cấu trúc if

https://docs.arduino.cc/language-reference/en/structure/control-structure/if/

Bài 5/6 · Hiệu chuẩn và sai số phép đo

Dùng dữ liệu đối chiếu để ước lượng lệch đo và kiểm chứng hiệu chỉnh trên dữ liệu mới.

Chuẩn bị

Đã đo khoảng cách có xử lý timeout; biết tính trung bình và lập bảng bằng cm.

Sau bài học, bạn có thể

1. Tính sai số có dấu và sai số tuyệt đối từ cặp đo–chuẩn.

2. Phân biệt lệch hệ thống với dao động ngẫu nhiên.

3. Kiểm chứng hiệu chỉnh trên vị trí chưa dùng để ước lượng offset.

Cùng hiểu bài từ đầu

Số hiện trên màn hình không tự trở thành sự thật. Hiệu chuẩn bắt đầu bằng việc so số đo với một mốc tham chiếu có cách đặt rõ ràng. Bạn sẽ học cách nhận ra một lệch gần cố định, sửa thử nó rồi kiểm tra ở vị trí mới.

Một chiếc cân luôn hiển thị 0,5 kg khi chưa đặt gì lên cân gợi ý một độ lệch ban đầu. Đặt vật lên nhiều lần và lấy trung bình vẫn có thể giữ nguyên độ lệch ấy.

Cảm biến khoảng cách cũng có thể đọc cao hơn hoặc thấp hơn mốc khá đều. Bù một hằng số là thử sửa phần lệch gần cố định; nó khác với làm mượt các dao động giữa các lượt.

Giới hạn của ví von: Không phải sai số nào cũng là lệch hằng số. Nếu sai số phụ thuộc khoảng cách, góc hoặc vật liệu, một phép trừ duy nhất sẽ không đủ; mốc tham chiếu cũng có độ không chắc chắn.

Những từ cần hiểu

1. Mốc tham chiếu · cm: Khoảng cách được xác định bằng thước và cách đặt đã ghi. Đây là cơ sở đối chiếu của thí nghiệm.

2. Sai số có dấu · e: e = số đo − mốc. Số dương nghĩa đo lớn hơn mốc; số âm nghĩa đo nhỏ hơn mốc.

3. Sai số tuyệt đối · |e|: Độ lớn của chênh lệch, luôn không âm. Ví dụ e = −0,8 cm cho |e| = 0,8 cm.

4. Offset · độ lệch bù: Hằng số ước lượng từ dữ liệu để dùng trong d_sửa = d_đo − offset. Phải kiểm chứng bằng dữ liệu mới.

Một phép bù có giúp ở điểm chưa dùng để tính không?

Bắt đầu với: Các cặp (mốc, đo) là (10; 10,8), (20; 20,9), (30; 30,7) cm. Dành riêng điểm mốc 25 cm, số đo 25,9 cm để kiểm chứng.

Tính sai số từng điểm học

Lấy đo trừ mốc: +0,8; +0,9; +0,7 cm. Các số đều dương và khá gần nhau, gợi ý thử mô hình lệch hằng số trong phạm vi này.

Ước lượng độ lệch bù

Trung bình ba sai số: offset = (0,8 + 0,9 + 0,7)/3 = 0,8 cm. Không dùng điểm 25 cm trong bước tính này.

Kiểm chứng bằng điểm mới

Ở mốc 25 cm, trước bù sai số là +0,9 cm. Sau bù: 25,9 − 0,8 = 25,1 cm, sai số còn +0,1 cm. Ghi rõ điều kiện thử và số mẫu lỗi nếu có.

Kết quả: Ở điểm kiểm chứng này, sai số tuyệt đối giảm từ 0,9 cm còn 0,1 cm. Đó là bằng chứng cải thiện tại điểm đã thử.

Tách dữ liệu ước lượng và kiểm chứng giúp tránh chỉ "sửa cho khớp" các số đã biết. Cần thêm vị trí và lượt đo để xem mô hình bù còn phù hợp ở đâu.

Tự giải thích rồi đối chiếu

Lấy trung bình 100 số cùng lệch +0,8 cm có tự xóa lệch đó không?

Không. Trung bình có thể giảm một phần dao động ngẫu nhiên trong điều kiện phù hợp, nhưng vẫn giữ lệch cố định khoảng +0,8 cm.

Nếu đọc thấp hơn mốc 1 cm thì offset và phép sửa có dấu gì?

offset gần −1 cm. Công thức d_sửa = d_đo − (−1) tương đương cộng 1 cm. Dấu phải theo quy ước đo trừ mốc.

Chọn độ sâu phù hợp

Mới bắt đầu · Lập bảng bằng phép trừ

Vẽ ba cột mốc, số đo, đo trừ mốc. Viết cm ở tiêu đề để mọi phép trừ dùng cùng đơn vị. Chưa cần công thức thống kê để nhận ra các sai số cùng cao hơn.

Giải thích bằng lời vì sao đọc cao hơn thì ta thử trừ một lượng dương.

Đã học cơ bản · Tách lệch và độ phân tán

Ở cùng một mốc, ghi nhiều lần để nhìn độ dao động. Ở nhiều mốc, nhìn xem sai số trung bình có gần cố định không. Hai cách thu thập trả lời hai câu hỏi khác nhau.

Đề xuất thêm hai vị trí kiểm chứng, cùng cách đặt vật và cách ghi timeout.

Muốn tìm hiểu sâu · Kiểm tra giới hạn của mô hình offset

Nếu sai số tăng dần khi mốc tăng, có thể cần mô hình có hệ số tỷ lệ thay vì chỉ bù hằng số. Không chọn mô hình phức tạp chỉ để mọi điểm đã đo khớp hoàn toàn; cần kiểm chứng độc lập và xét độ không chắc chắn của mốc.

So sánh tập sai số +0,5; +1,0; +1,5 cm tại 10; 20; 30 cm với giả thuyết lệch hằng số. Nêu dữ liệu bổ sung cần có trước khi chọn cách sửa.

Kiến thức trọng tâm

Phép đo cần một mốc đối chiếu

Chọn d_chuẩn từ thước tham chiếu phù hợp và ghi điều kiện đo. Với mỗi mẫu hợp lệ: e = d_đo − d_chuẩn (cm); |e| là sai số tuyệt đối. Dấu của e cho biết đang đo lớn hay nhỏ hơn mốc, không cho biết mẫu đó đáng tin đến mức nào.

Lệch và nhiễu không giống nhau

Nếu nhiều số đo đều cao hơn khoảng 1 cm, có thể có lệch hệ thống. Nếu kết quả dao động quanh mốc, có thành phần ngẫu nhiên. Trung bình nhiều mẫu có thể giảm ảnh hưởng dao động trong một số điều kiện, nhưng không tự sửa lệch cố định. Không thay timeout bằng 0; cần ghi số mẫu hợp lệ và số mẫu lỗi.

Ước lượng rồi kiểm chứng

Mô hình bù đơn giản d_sửa = d_đo − offset chỉ phù hợp khi dữ liệu gợi ý lệch gần cố định trong phạm vi khảo sát. Ước lượng offset từ một tập dữ liệu; kiểm tra trên vị trí mới chưa dùng để tính offset. Nếu sai số tăng theo khoảng cách hoặc thay đổi theo góc, chỉ bù hằng số có thể không đủ; không ép mô hình để mọi số đều đẹp.

Ví dụ có lời giải

Ba cặp (chuẩn,đo) = (10,11), (20,21), (30,31) cm cho e = +1,+1,+1 cm, offset = +1 cm. Ở điểm kiểm chứng mới chuẩn 25 cm, đo 26,2 cm: trước bù |e| = 1,2 cm; sau bù d_sửa = 25,2 cm và |e| = 0,2 cm. Một điểm tốt chưa đủ chứng minh đúng trên mọi khoảng cách.

Lỗi thường gặp và cách tránh

1. Hiệu chuẩn không phải xóa những mẫu bất lợi rồi báo độ chính xác cao.

2. Không dùng lại duy nhất chính các điểm đã khớp để kết luận mô hình đúng trên dữ liệu mới.

Các bước thực hành

1. Tại 10, 20 và 30 cm, thu ít nhất ba mẫu mỗi vị trí và ghi điều kiện đặt vật.

2. Tách mẫu timeout khỏi số liệu; ghi tỷ lệ mẫu hợp lệ.

3. Tính e từng mẫu, trung bình e và quan sát mức dao động ở mỗi vị trí.

4. Chỉ thử bù offset nếu bảng dữ liệu ủng hộ lệch gần cố định.

5. Kiểm chứng ở 25 cm hoặc vị trí mới khác, báo sai số trước/sau cả khi kết quả xấu đi.

Mã mẫu · Arduino C++ · sketch xử lý bảng số liệu trên Serial

```
// Vi du xu ly so lieu da thu, khong tu doc cam bien.
void setup() {
 Serial.begin(9600);
 const float reference[] = {10, 20, 30};
 const float measured[] = {11, 21, 31};
 float offset = 0;
 for (int i = 0; i < 3; ++i) offset += measured[i] - reference[i];
 offset /= 3.0f;
 float newMeasured = 26.2f; // Mau moi, moc chuan 25 cm
 float corrected = newMeasured - offset;
 Serial.print("Offset cm: "); Serial.println(offset);
 Serial.print("Corrected cm: "); Serial.println(corrected);
}
void loop() {}
```

Chọn và sử dụng phòng lab

Thu dữ liệu từ môi trường hỗ trợ HC-SR04 hoặc dùng bảng số liệu giả định có ghi rõ. Sketch mẫu tính và in offset trong Serial từ số liệu đã khai báo. Để hiệu chuẩn kit thật, thu số đo cảm biến rồi kiểm chứng kết quả bù tại các mốc mới.

Bài tập áp dụng

Dùng dữ liệu (10,11), (20,21), (30,31) để tìm offset. Với điểm mới chuẩn 25 cm đo 26,2 cm, tính sai số tuyệt đối trước và sau bù. Nêu một tình huống bù hằng số không phù hợp.

Gợi ý đối chiếu sau khi tự làm

Offset = +1 cm; trước bù sai số tuyệt đối 1,2 cm; sau bù 0,2 cm. Nếu độ lệch thay đổi theo khoảng cách, nhiệt độ hoặc góc phản xạ thì không có cơ sở khẳng định một offset duy nhất phù hợp mọi trường hợp.

Tự kiểm tra mức hiểu

1. Nếu mọi số đo đều thấp hơn mốc 2 cm, offset có dấu gì và phải bù thế nào?

2. Trung bình 100 phép đo lệch cố định +1 cm có tự loại lệch đó không?

Câu hỏi trắc nghiệm

Tập dữ liệu kiểm chứng phải...

1. Trùng hoàn toàn tập hiệu chuẩn

2. Có điểm mới chưa dùng để hiệu chuẩn

3. Chỉ chứa kết quả tốt

Đáp án: 2. Điểm mới chưa dùng để tính offset giúp kiểm tra khả năng áp dụng của hiệu chỉnh; chỉ lặp lại dữ liệu cũ dễ tạo kết luận quá lạc quan.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

Arduino — pulseIn và giá trị timeout

https://docs.arduino.cc/language-reference/en/functions/advanced-io/pulseIn/

NIST — tổng quan hiệu chuẩn và mô hình đo

https://www.itl.nist.gov/div898/handbook/mpc/section3/mpc3.htm

Bài 6/6 · Dự án: cảnh báo khoảng cách ổn định

Thiết kế cảnh báo hai ngưỡng, kiểm tra trạng thái trước đó và chính sách khi mất số đo.

Chuẩn bị

Đã học cảnh báo một ngưỡng, dữ liệu hợp lệ và kiểm thử biên; biết biến bool lưu trạng thái.

Sau bài học, bạn có thể

1. Vẽ được sơ đồ hai trạng thái OFF/ON với hai ngưỡng riêng.

2. Dự đoán chuỗi đầu ra có xét trạng thái trước đó.

3. Phân biệt hysteresis với làm mượt số đo và xử lý lỗi.

Cùng hiểu bài từ đầu

Một vật đứng gần ranh giới có thể làm số đo nhích lên xuống và đèn chớp liên tục. Để tránh đổi quyết định quá dễ, chương trình có thể nhớ trạng thái trước đó và dùng hai ngưỡng khác nhau cho việc bật và tắt. Cách này gọi là hysteresis.

Máy điều hòa có thể bật làm mát khi phòng nóng hơn một mức và chỉ tắt khi đã mát xuống một mức khác. Ở khoảng giữa, nó giữ quyết định hiện tại.

Cảnh báo cũng có một vùng giữ trạng thái: dưới 20 cm bật, trên 25 cm tắt, từ 20 đến 25 cm giữ như trước. Cùng 22 cm có thể cho ON hoặc OFF tùy lịch sử.

Giới hạn của ví von: Quy tắc hai ngưỡng không làm cảm biến đo chính xác hơn. Bài cảnh báo cũng không tự có đầy đủ cơ chế, độ trễ và bảo vệ của một thiết bị điều nhiệt thực.

Những từ cần hiểu

1. Trạng thái nhớ: Một giá trị được giữ từ lượt trước, ở đây là cảnh báo ON hoặc OFF.

2. Hysteresis · hai ngưỡng: Bật và tắt dùng hai ranh giới khác nhau; đầu ra còn phụ thuộc trạng thái trước đó.

3. Vùng giữ · 20–25 cm: Với dấu < 20 và > 25 của bài, cả đúng 20 và đúng 25 đều giữ trạng thái cũ.

4. Lọc dữ liệu: Xử lý chuỗi số đo để giảm một số dao động. Khác với hysteresis, vốn thay cách ra quyết định và cần bộ nhớ trạng thái.

Theo dõi trạng thái nhớ và LED qua một lượt lỗi

Bắt đầu với: Ban đầu trạng thái nhớ OFF. Lần lượt nhận 28 cm, 19 cm, 22 cm, timeout, 25 cm, 26 cm. Chính sách lỗi của bài: giữ trạng thái nhớ, bật LED và ghi SENSOR_ERROR.

Khởi tạo và đi vào vùng gần

28 > 25 nên trạng thái OFF. Tiếp theo 19 < 20 nên chuyển ON. LED đi theo trạng thái ở hai lượt hợp lệ này.

Đi vào vùng giữ

22 nằm giữa 20 và 25 nên giữ ON. Không lấy việc 22 > 20 làm lý do tắt: đó sẽ là quy tắc một ngưỡng khác.

Tách lỗi khỏi cập nhật trạng thái

Timeout không cung cấp d mới. Giữ bộ nhớ ON và bật LED theo chính sách lỗi. Khi có lại số đo 25 cm, vẫn giữ ON vì điều kiện tắt là > 25, không phải ≥ 25.

Thoát khỏi vùng giữ

26 > 25 nên chuyển OFF và tắt LED. Sau đó nếu trở về 22 cm, trạng thái vẫn OFF cho tới khi có số đo dưới 20.

Kết quả: Trạng thái nhớ qua chuỗi là OFF, ON, ON, ON, ON, OFF. Lượt timeout cần kèm nhãn lỗi; không được báo rằng đã đo được vật gần.

Bạn cần ghi cả số đo hợp lệ, trạng thái nhớ và biểu hiện LED. Nếu timeout xảy ra khi bộ nhớ đang OFF thì LED vẫn bật do chính sách lỗi, dù bộ nhớ không đổi.

Tự giải thích rồi đối chiếu

Ở 22 cm, có thể xác định đầu ra chỉ từ số 22 không?

Không. Trong vùng giữ cần biết trạng thái trước. ON thì tiếp tục ON; OFF thì tiếp tục OFF.

Hai ngưỡng có loại bỏ sai số đo không?

Không. Chúng giảm một số chuyển trạng thái do dao động quanh một ranh giới; số đo đầu vào vẫn có sai số. Lọc, hiệu chuẩn và hysteresis giải quyết các vấn đề khác nhau.

Chọn độ sâu phù hợp

Mới bắt đầu · Dùng một thẻ nhớ ON/OFF

Viết ON ở một mặt giấy, OFF ở mặt kia. Đọc từng đầu vào và chỉ lật khi gặp điều kiện đổi rõ ràng. Giữ nguyên thẻ khi số đo trong vùng giữa.

Thử chuỗi 30, 22, 19, 22, 26 và đọc to lý do ở mỗi lượt.

Đã học cơ bản · So sánh hai quy tắc trên cùng chuỗi

Lập hai cột cho quy tắc một ngưỡng và hai ngưỡng. Đếm số lần đổi trạng thái trên cùng số đo; không đổi dữ liệu để làm một phương án đẹp hơn.

Kiểm thử riêng 20 cm và 25 cm với cả trạng thái đầu ON lẫn OFF.

Muốn tìm hiểu sâu · Đánh đổi ổn định và phản hồi

Tăng khoảng cách hai ngưỡng có thể giảm đổi qua lại nhưng cũng làm trạng thái cũ được giữ trong một vùng rộng hơn. Kết hợp lọc còn có thể tăng độ trễ. Chọn tham số cần dựa vào nhiệm vụ, số đo và tiêu chí, không chỉ số lần nhấp nháy ít nhất.

Đề xuất tiêu chí so sánh độ ổn định và thời gian phản hồi; mô tả cách kiểm thử timeout khi bộ nhớ đang OFF rồi phục hồi ở 22 cm.

Kiến thức trọng tâm

Vì sao một ngưỡng dễ rung trạng thái

Nếu số đo dao động 19,9 ↔ 20,1 cm, quy tắc d < 20 có thể làm đèn bật/tắt nhiều lần. Không phải mọi thay đổi của đầu ra đều phản ánh vật đang di chuyển nhiều; nhiễu đo cũng có thể gây chuyển trạng thái.

Hysteresis nhớ trạng thái

Quy tắc: d < 20 cm thì bật; d > 25 cm thì tắt; từ 20 đến 25 cm, kể cả hai đầu mút, giữ trạng thái trước đó. Cần khởi tạo trạng thái rõ ràng. Hysteresis làm ổn định quyết định bằng hai ngưỡng, không tự sửa số đo. Lọc trung bình tác động lên dữ liệu và có thể gây trễ; hai kỹ thuật khác nhau.

Thiết kế lỗi và tiêu chí đánh giá

Ví dụ này khởi tạo cảnh báo OFF; nếu mất Echo thì báo SENSOR_ERROR, giữ trạng thái nhớ và bật LED để gây chú ý. Khi có số đo hợp lệ trở lại, cập nhật theo hai ngưỡng. Đây là chính sách của bài học, phải được mô tả và kiểm thử; không phải chứng nhận an toàn. Đánh giá bằng số lần chuyển trạng thái, hành vi ở biên và dữ liệu nhật ký, không chỉ một lần chạy thuận lợi.

Ví dụ có lời giải

Ban đầu OFF. Chuỗi d = 30,19,22,24,26 cm tạo OFF,ON,ON,ON,OFF. Với chuỗi 19,9;20,1;19,8;20,2, quy tắc một ngưỡng đổi trạng thái liên tục, còn hai ngưỡng sẽ giữ ON sau lần đầu bật. Ở đúng 20 hoặc 25 cm phải giữ trạng thái trước đó.

Lỗi thường gặp và cách tránh

1. Trong vùng 20–25 cm không có một đầu ra duy nhất nếu chưa biết trạng thái trước đó.

2. Hysteresis không loại nhiễu khỏi phép đo, không thay thế xử lý timeout và không bảo đảm robot dừng kịp.

Các bước thực hành

1. Vẽ hai nút OFF/ON và hai mũi tên có điều kiện; viết quy tắc khởi động và lỗi.

2. Dự đoán từng trạng thái của hai chuỗi số liệu trước khi chạy.

3. Chạy sketch đầy đủ; thử các điểm sát 20 cm và sát 25 cm.

4. Bổ sung timeout rồi phục hồi; ghi riêng LED cảnh báo lỗi và trạng thái nhớ alarm.

5. So sánh số lần chuyển với bản một ngưỡng, nêu lợi ích và khoảng giữ trạng thái gây ra.

Mã mẫu · Arduino C++ · sketch hai ngưỡng cho Uno + HC-SR04

```
const byte TRIG = 9;
const byte ECHO = 10;

bool readDistance(float &cm) {
 digitalWrite(TRIG, LOW); delayMicroseconds(2);
 digitalWrite(TRIG, HIGH); delayMicroseconds(10);
 digitalWrite(TRIG, LOW);
 unsigned long us = pulseIn(ECHO, HIGH, 30000UL);
 if (us == 0) return false;
 cm = us * 0.034f / 2.0f;
 return true;
}

void setup() {
 Serial.begin(9600);
 pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
 pinMode(LED_BUILTIN, OUTPUT);
}

bool alarm = false; // Trang thai nho khi co so do hop le
void loop() {
 float cm = 0;
 bool valid = readDistance(cm);
 if (!valid) {
  digitalWrite(LED_BUILTIN, HIGH); // Bao loi, khong sua bien alarm
  Serial.println("SENSOR_ERROR");
 } else {
  if (cm < 20.0f) alarm = true;
  else if (cm > 25.0f) alarm = false;
  digitalWrite(LED_BUILTIN, alarm ? HIGH : LOW);
  Serial.print(cm); Serial.println(alarm ? " cm ON" : " cm OFF");
 }
 delay(100);
}
```

Chọn và sử dụng phòng lab

Dùng chuỗi khoảng cách trong bàn thí nghiệm để theo dõi trạng thái qua hai ngưỡng 20/25 cm. Đối chiếu kết quả với bảng OFF/ON và sketch của bài. Kiểm tra thêm trường hợp mất Echo trong môi trường có cảm biến hoặc bằng bộ dữ liệu lỗi được ghi rõ.

Bài tập áp dụng

Lập bảng trạng thái với khởi tạo OFF cho chuỗi 30,19,20,22,25,26 cm. Thêm một mẫu lỗi giữa 22 và 25 cm; mô tả riêng trạng thái nhớ và đầu ra báo lỗi.

Gợi ý đối chiếu sau khi tự làm

Với tất cả mẫu hợp lệ: OFF,ON,ON,ON,ON,OFF. Mẫu lỗi giữ biến alarm trước đó và bật LED báo chú ý trong chính sách code mẫu; không biến mẫu lỗi thành 0 cm. Tại 20 và 25 cm, cả hai điều kiện chuyển trạng thái đều sai nên giữ trạng thái.

Tự kiểm tra mức hiểu

1. Đang OFF rồi nhận 22 cm thì kết quả khác gì đang ON rồi nhận 22 cm?

2. Sau OFF → lỗi đo → 22 cm, LED và biến alarm trong code mẫu có thể khác nhau ở những thời điểm nào?

Câu hỏi trắc nghiệm

Đang bật, khoảng cách bằng 22 cm thì...

1. Tắt

2. Giữ bật

3. Bắt buộc nhấp nháy

Đáp án: 2. Khi đang bật, 22 cm nằm giữa hai ngưỡng. Nó không nhỏ hơn 20 để đặt lại ON cũng không lớn hơn 25 để đặt OFF, nên biến alarm giữ giá trị true.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

Arduino — pulseIn và giá trị timeout

https://docs.arduino.cc/language-reference/en/functions/advanced-io/pulseIn/

Arduino — Blink Without Delay

https://docs.arduino.cc/built-in-examples/digital/BlinkWithoutDelay/